

SP ENGINEERING ACADEMY

Video Lectures / Course Materials

COMPLETE COURSE

DIGITAL SYSTEMS

DIGITAL CIRCUIT DESIGN

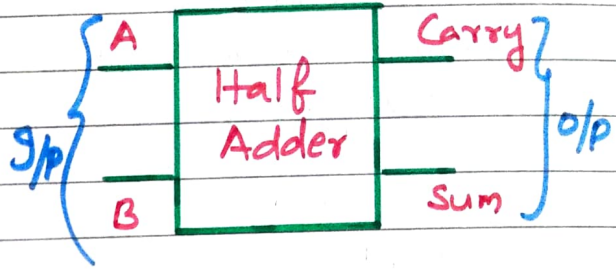
Part – VIII – Digital Adders and Subtractors

Topic #	Content	
91	Half Adder - Truth Table, Logic Circuit and Explanation - https://youtu.be/sFOkgt9mNc	02
92	Design of Half Adder Logic Circuit using NAND Gate - https://youtu.be/0Ai_b-OxWpM	03
93	Full Adder - Truth Table, Logic Circuit and Explanation - https://youtu.be/nl0xVBZPe5s	06
94	Design of Full Adder using Two Half Adders - https://youtu.be/3j4_4rIJKw	04
95	Half Subtractor - Truth Table, Logic Circuit and Explanation - https://youtu.be/qfoWUV5J8yk	05
96	Design of Half Subtractor Logic Circuit using NAND Gate - https://youtu.be/v47SvnF41ok	07
97	Full Subtractor - Truth Table, Logic Circuit and Explanation - https://youtu.be/P5lOH1QSxuQ	08
98	Design of Full Subtractor using Two Half Subtractors - https://youtu.be/2jQv-GzPIQo	09
99	BCD Adder - Truth Table, Logic Circuit and Explanation - https://youtu.be/Rbxnj-1cB20	11
100	Design of 8 Bit BDC adder using 4 Bit BCD Adders - https://youtu.be/g40rxGTCxM4	14
101	Design of BCD to Excess - 3 Code Converter using 4 Bit BCD Adder - https://youtu.be/l5ytGGNoLJ4	14
102	Carry Look Ahead Adder - Logic Circuit and Explanation - https://youtu.be/WSOPhwzgHV8	15

① Half Adder

The half adder operation needs two binary inputs : augend and addend bits and two binary outputs : Sum and Carry

Inputs		Outputs	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



K-Map Simplification

For Carry

A \ B	0	1
0		
1		1

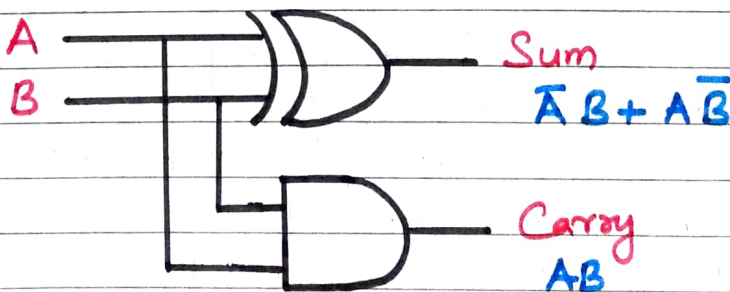
For Sum

A \ B	0	1
0		1
1	1	

$$\therefore \text{Carry} = AB$$

$$\therefore \text{Sum} = A\bar{B} + \bar{A}B$$

$$\text{Sum} = A \oplus B$$



Limitations:

In multidigit addition we have to add two bits along with the Carry of previous digit addition. Effectively such addition of three bits. This is not possible with half-adder. Hence half-adder are not used in practice.

Example

Video Link - https://youtu.be/0Ai_b-OxWpM

Draw the half adder using NAND gates

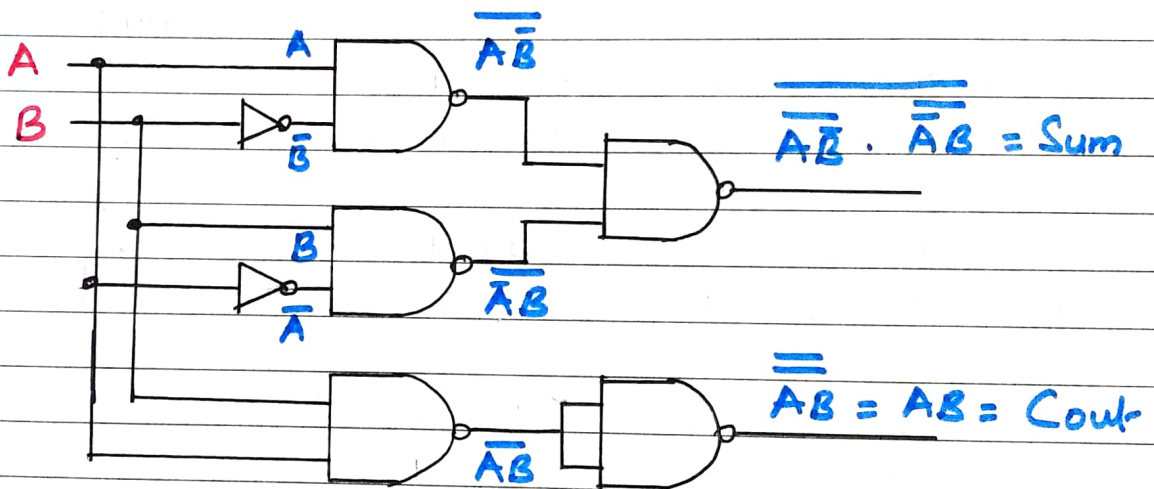
For half adder

$$\text{Sum} = A\bar{B} + \bar{A}B$$

$$= \overline{\overline{A\bar{B}} + \overline{\bar{A}B}}$$

$$= (\overline{A\bar{B}}) \cdot (\overline{\bar{A}B})$$

$$\text{Cout} = AB$$



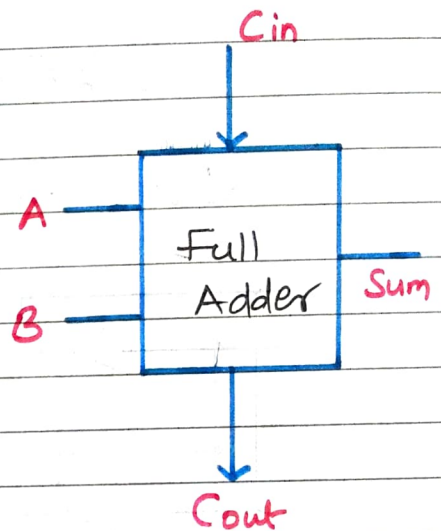
② Full Adder

Video Link - <https://youtu.be/nl0xVBZPe5s>

A full adder is a Combinational Circuit that forms the arithmetic sum of three input bits. It consists of three inputs and two outputs.

Truth Table for Full adder

Inputs			Outputs	
A	B	Cin	Carry	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



For Carry

A \ BCin	00	01	11	10
0			1	
1		1	1	1

For Sum

A \ BCin	00	01	11	10
0		1		1
1	1		1	

$$Cout = AB + ACin + BCin$$

$$Sum = \bar{A}\bar{B}Cin + \bar{A}B\bar{Cin}$$

$$+ A\bar{B}\bar{Cin} + ABCin$$

The Boolean function for Sum can be further simplified as

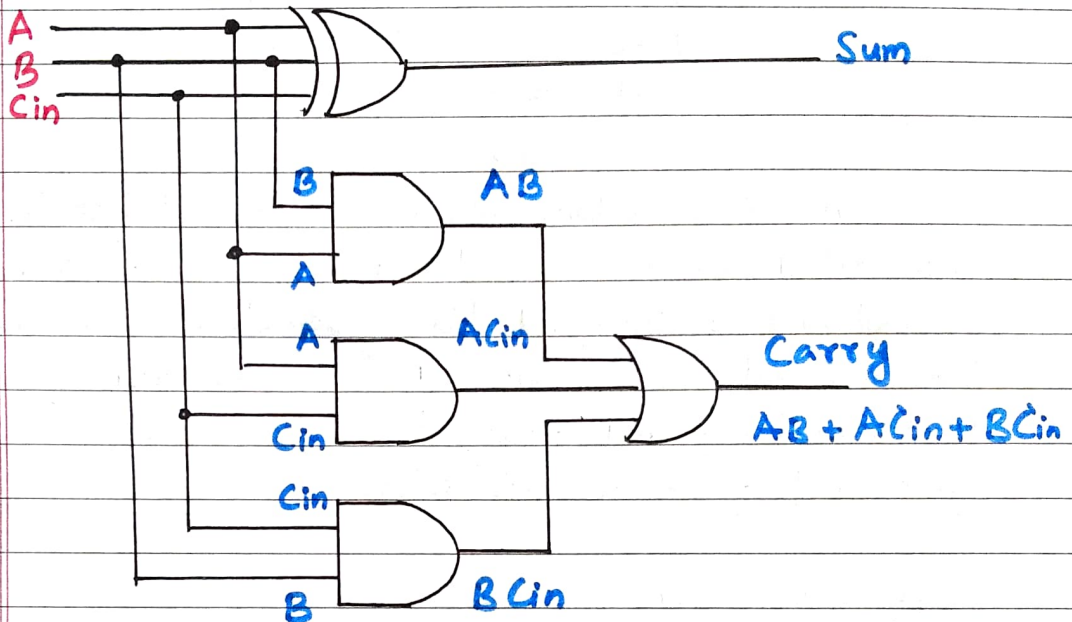
$$Sum = \bar{A}\bar{B}Cin + \bar{A}B\bar{Cin} + A\bar{B}\bar{Cin} + ABCin$$

$$= Cin(\bar{A}\bar{B} + AB) + \bar{Cin}(\bar{A}B + A\bar{B})$$

$$= Cin(\bar{A} \odot B) + \bar{Cin}(A \oplus B)$$

$$\text{Sum} = \text{Cin} (\overline{A \oplus B}) + \overline{\text{Cin}} (A \oplus B)$$

$$\text{Sum} = \text{Cin} \oplus (A \oplus B)$$



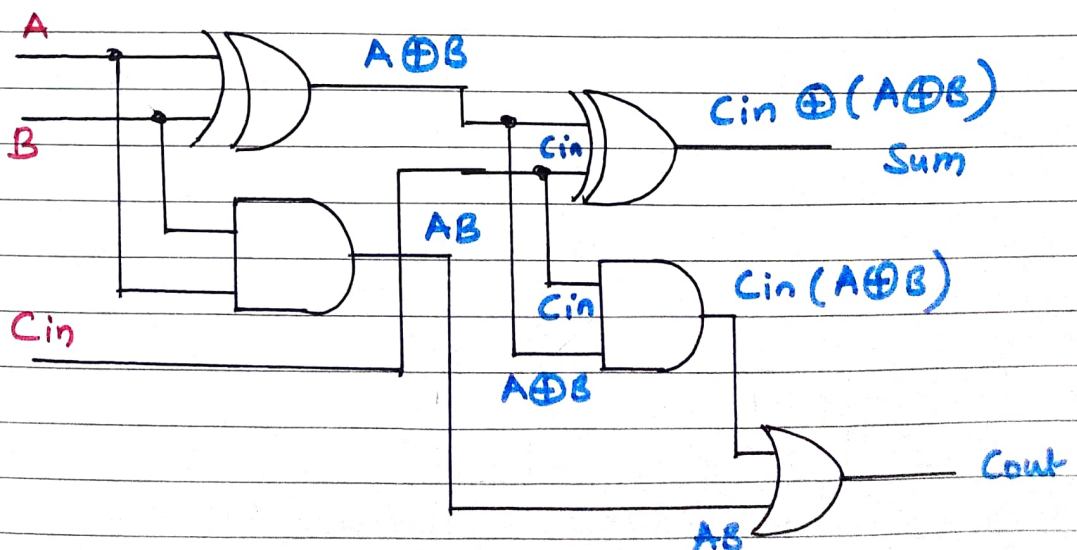
Video Link - https://youtu.be/3j4_4rIIJKw

Full adder Using two half adders

$$\begin{aligned} \text{Cout} &= AB + ACin + BCin = AB + ACin(B + \bar{B}) + BCin(A + \bar{A}) \\ &= AB + ACinB + ACin\bar{B} + BCinA + BCin\bar{A} \\ &= AB(1 + Cin + Cin) + A\bar{B}Cin + \bar{A}BCin \\ &= AB + Cin(A\bar{B} + \bar{A}B) \\ \text{Cout} &= AB + Cin(A \oplus B) \end{aligned}$$

First Half Adder

Second Half Adder



Subtractors

The subtraction consists of four possible elementary operations,

$$0 - 0 = 0$$

$$0 - 1 = 1 \text{ with borrow}$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

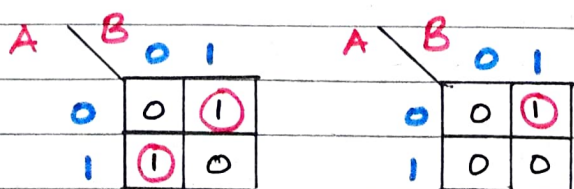
In all operations, each subtrahend bit is subtracted from minuend bit. In case of second operation the minuend bit is smaller than the subtrahend bit, hence 1 is borrowed. Just as there half and full adders, there are half and full subtractors

Half Subtractor

Video Link - <https://youtu.be/qfoWUV5J8yk>

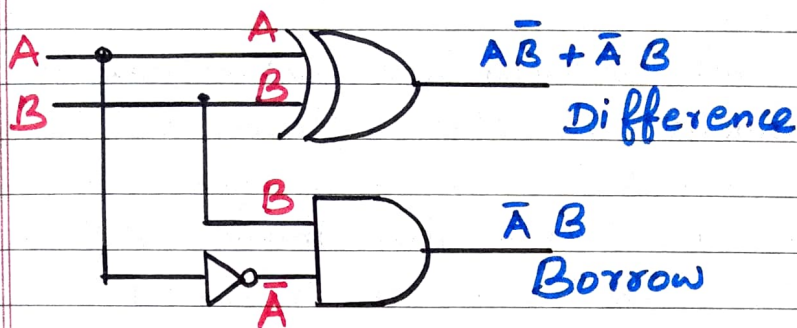
A half Subtractor is a Combinational Circuit that subtracts two-bits and produces their difference. It also has an output to specify if a 1 has been borrowed. Let us designate minuend bit as A and the subtrahend bit as B. The result of operation $A - B$ for all possible values of A and B is tabulated below

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



$$\begin{aligned} \text{Difference} &= A\bar{B} + \bar{A}B \\ &= A \oplus B \end{aligned}$$

$$\text{Borrow} = \bar{A}B$$



Limitations of half-Subtractor

In multidigit subtraction, we have to subtract two bits along with the borrow of previous digit subtraction. Effectively such subtraction requires subtraction of three bits. This is not possible with half-subtractor.

Example

Video Link - <https://youtu.be/v47SvnF41ok>

Draw the half Subtractor using NAND gates

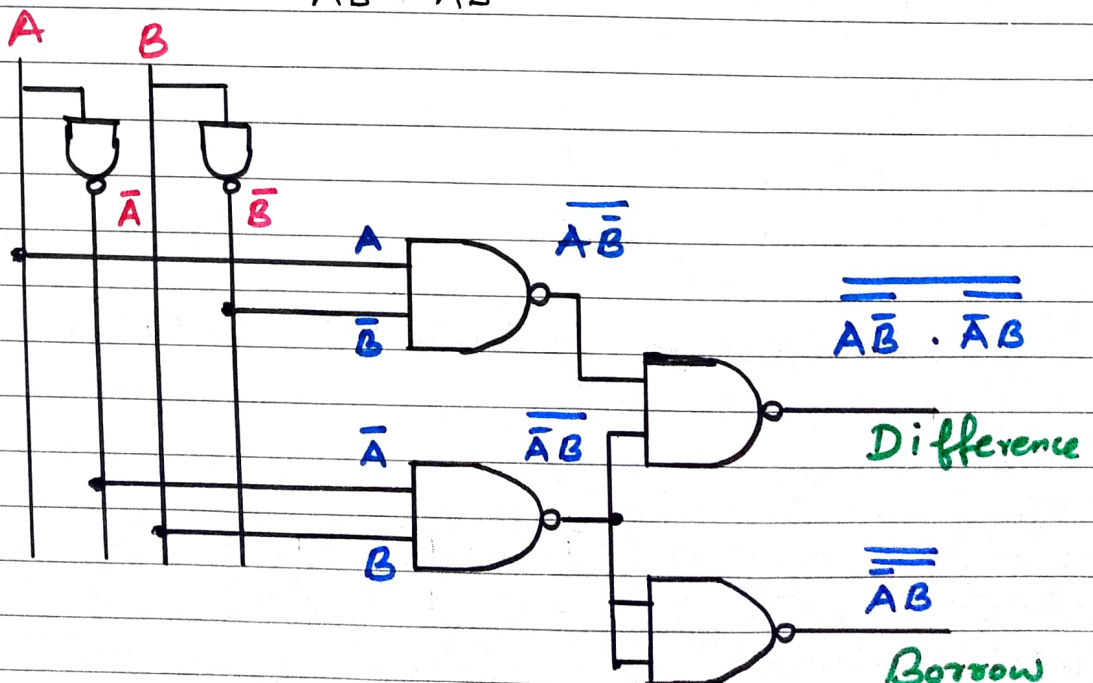
$$\text{Difference} = A\bar{B} + \bar{A}B$$

$$\text{Borrow} = \bar{A}B$$

$$= \overline{\overline{A\bar{B} + \bar{A}B}}$$

$$= \overline{\overline{\bar{A}B}}$$

$$= \overline{\bar{A}\bar{B}} \cdot \overline{\bar{A}B}$$



Full Subtractor

Video Link - <https://youtu.be/P5lOH1QSxuQ>

A full subtractor is a Combinational Circuit that performs a Subtraction between two bits, taking into account borrow of the lower significant stage. This Circuit has three inputs and two outputs. The three inputs are A, B and Bin, denote the minuend, Subtrahend and previous borrow respectively. The two outputs D and Bout represent the difference and output borrow, respectively. The truth table is shown below

Inputs			Outputs	
A	B	Bin	Difference D	Bout Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

A	B Bin			
	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$\text{Difference } D = \bar{A}\bar{B}B_{in} + \bar{A}B\bar{B}_{in} + A\bar{B}\bar{B}_{in} + AB B_{in}$$

A	B Bin			
	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$\text{Borrow } Bout = \bar{A}B_{in} + \bar{A}B + B B_{in}$$

The Boolean function for D (Difference) can be further simplified as

$$D = \bar{A} \bar{B} B_{in} + \bar{A} B \bar{B}_{in} + A \bar{B} \bar{B}_{in} + A B B_{in}$$

$$= B_{in} (\bar{A} \bar{B} + A B) + \bar{B}_{in} (\bar{A} B + A \bar{B})$$

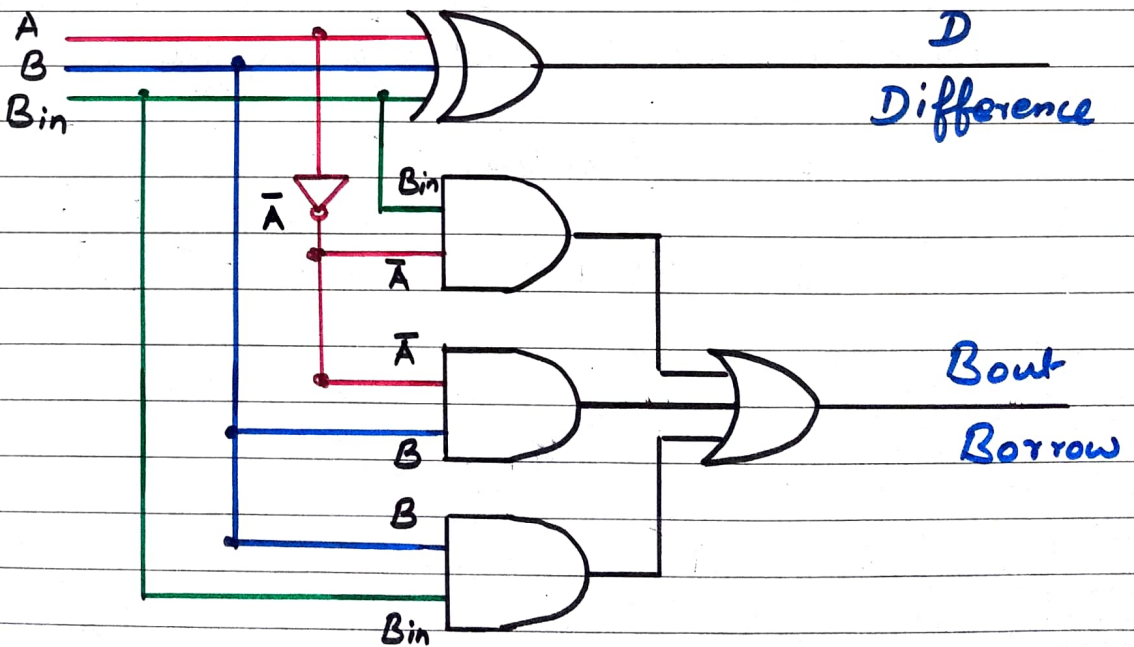
$$= B_{in} (A \odot B) + \bar{B}_{in} (A \oplus B)$$

$$= B_{in} (\overline{A \oplus B}) + \bar{B}_{in} (A \oplus B)$$

$$D = B_{in} \oplus (A \oplus B)$$

$$B_{out} = \bar{A} B_{in} + \bar{A} B + B B_{in}$$

With this simplified Boolean function, Circuit for full-Subtractor can be implemented as follows



The borrow output for circuit shown, can be given as

$$B_{out} = \bar{A} B_{in} + \bar{A} B + B B_{in}$$

$$= \bar{A} B_{in} (B + \bar{B}) + \bar{A} B + B B_{in} (A + \bar{A})$$

$$= \bar{A} B B_{in} + \bar{A} \bar{B} B_{in} + \bar{A} B + A B B_{in} + \bar{A} B B_{in}$$

$$= \bar{A} B (B_{in} + 1 + B_{in}) + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + B_{in} (\bar{A} \bar{B} + A B)$$

$$B_{out} = \bar{A} B + B_{in} (\overline{A \oplus B})$$

A full subtractor can also be implemented with two half subtractors and one OR gate

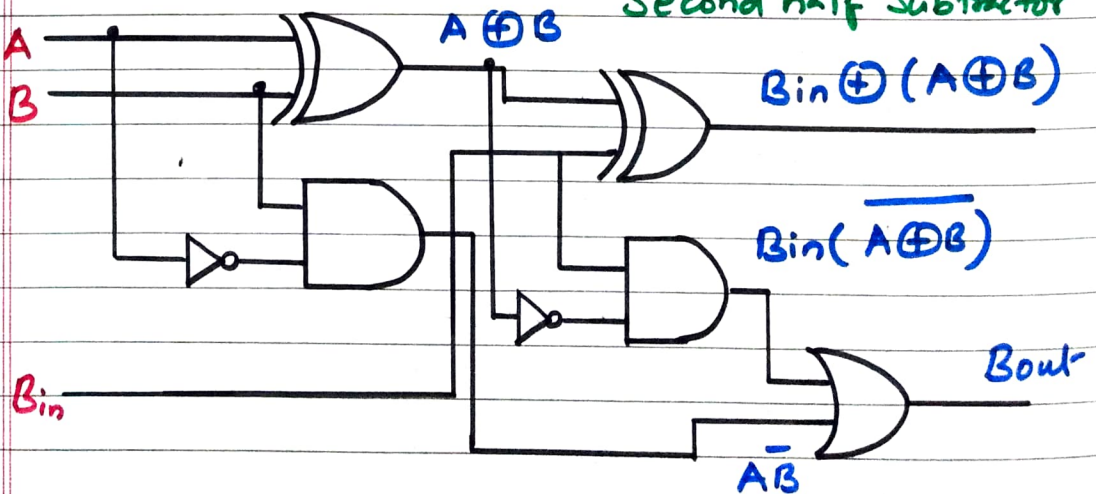
$$D = B_{in} \oplus (A \oplus B)$$

$$B_{out} = \bar{A} B + B_{in} (\overline{A \oplus B})$$

First Half Subtractor

Video Link - <https://youtu.be/2jQv-GzPIQo>

Second half Subtractor



BCD Adder

Video Link - <https://youtu.be/Rbxnj-1cB20>

The digital Systems handle the decimal number in the form of Binary Coded Decimal (BCD). A BCD adder is a circuit that adds two BCD digits and produces a sum digit also in BCD. Here implementation of BCD adder as follows

- 4-bit binary adder for initial addition
- Logic circuit to detect sum greater than 9
- One more 4-bit adder to add 0110 in the sum if sum is greater than 9 or Carry is 1

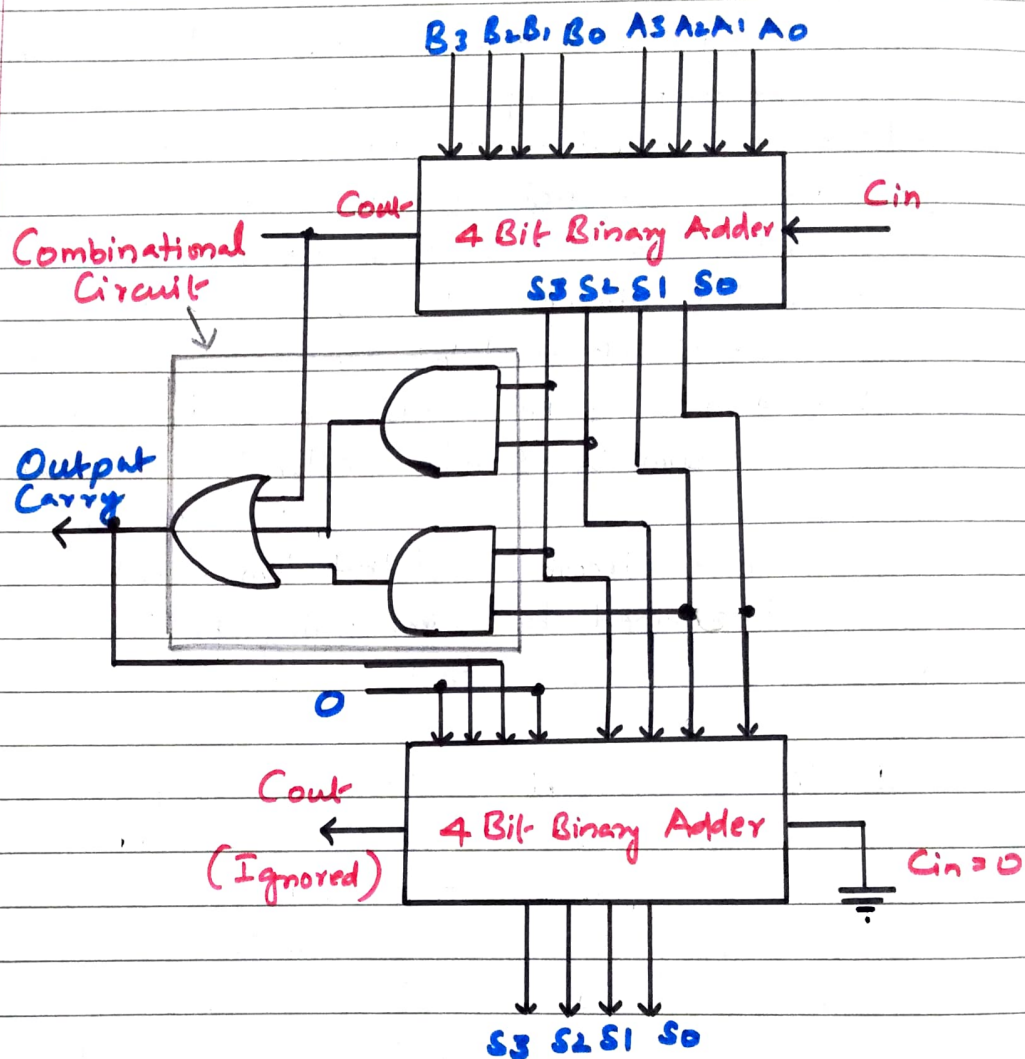
The logic circuit to detect sum greater than 9 can be determined by simplifying the Boolean expression of given truth table

Inputs	Output
$S_3 S_2 S_1 S_0$	Y
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	0
0 1 0 0	0
0 1 0 1	0
0 1 1 0	0
0 1 1 1	0
1 0 0 0	0
1 0 0 1	0
1 0 1 0	1
1 0 1 1	1
1 1 0 0	1
1 1 0 1	1
1 1 1 0	1
1 1 1 1	1

$Y = 1$ indicates sum is greater than 9. we can put one more term, Cout in the above expression to check whether Carry is One. If any one condition is satisfied we add 6 (0110) in the sum

$S_3 S_2$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

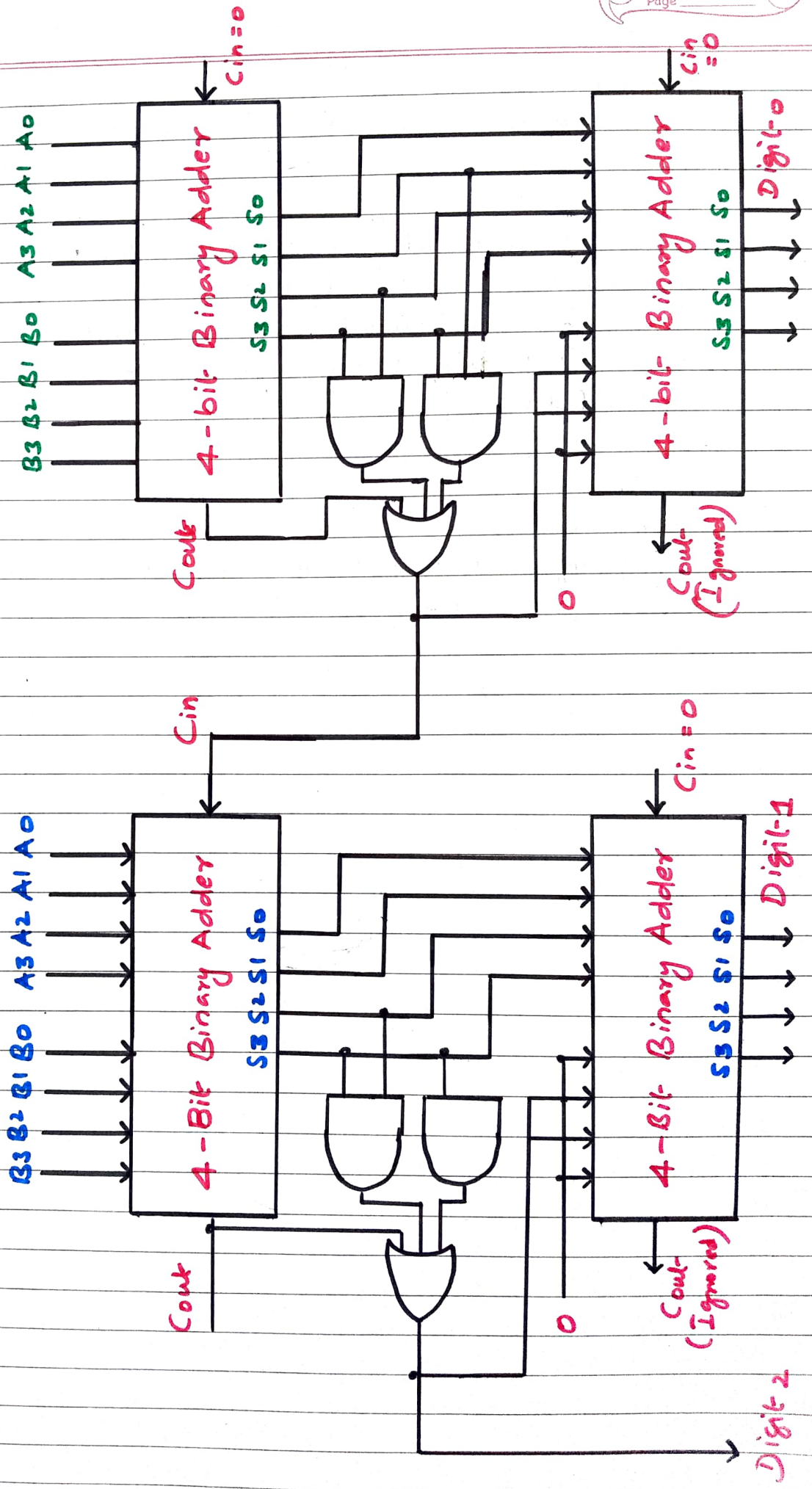
$$Y = S_3 S_2 + S_3 S_1$$



As shown in diagram, the two BCD numbers, together with inputs carry, are first added in the top 4-bit binary adder to produce a binary sum

When the output carry is equal to zero, nothing is added to the binary sum. when it is equal to one (when $\text{sum} > 9$ or $\text{Cout} = 1$), binary 0110 is added to the binary sum through the bottom 4-bit binary adder

The output carry generated from the bottom binary adder can be ignored, since it supplies information already available at the output - carry terminal



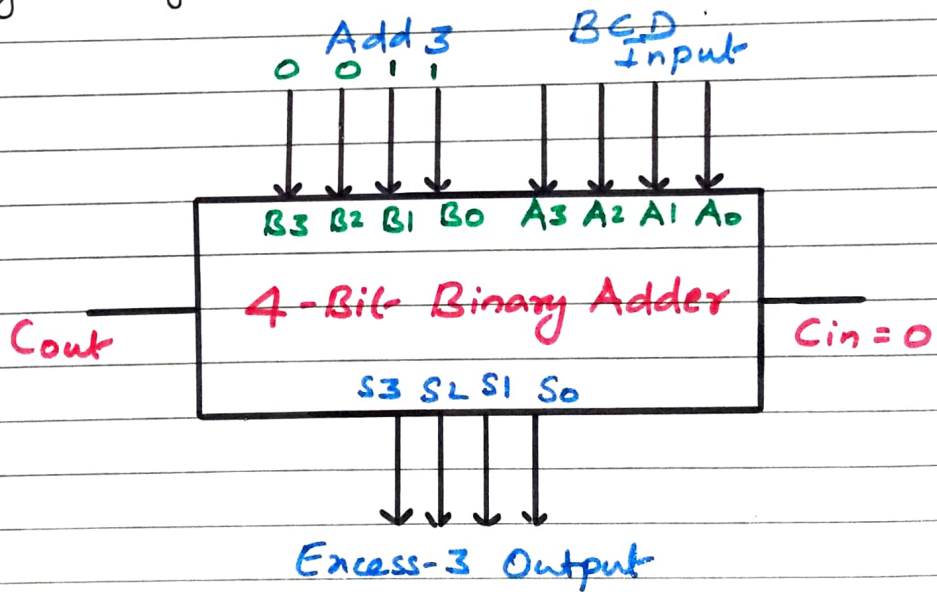
Example 01

Design an 8-bit BCD adder using 4-bit adder

To implement 8-bit BCD adder we have to cascade two 4-bit BCD adder. In Cascade Connection Carry output of the lower position (digit) is connected as a Carry input of the higher position (digit).

Example 02

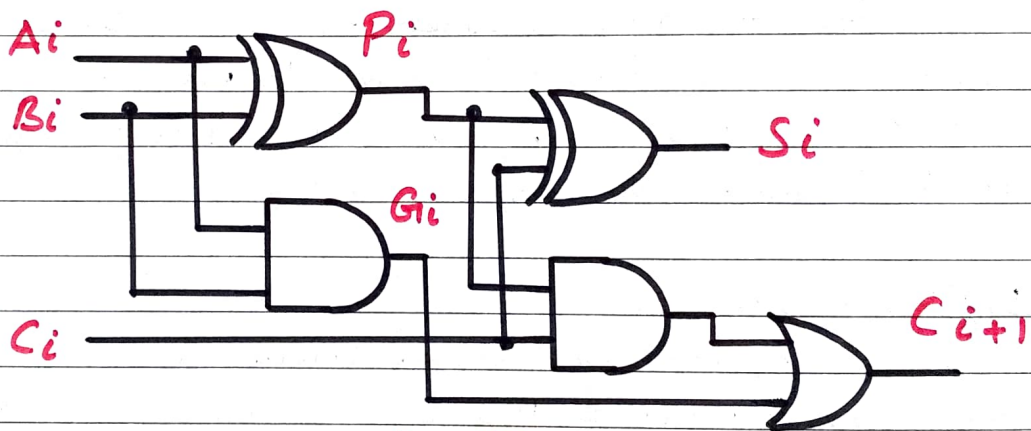
Design a BCD to Excess-3 Code Converter using binary parallel adder



Carry Look Ahead Adder Video Link - <https://youtu.be/WSOPhwzgHV8>

One method of speeding up parallel addition process by eliminating inter stage carry delay is called look ahead - carry addition. This method utilizes logic gates to look at the lower-order bits of augend and addend to see if a higher-order carry is to be generated. It uses two functions: Carry generate and Carry propagate.

Consider the circuit of the full adder. Here two functions: Carry generate and Carry propagate.



$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

The output Sum and Carry Can be expressed as

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

G_i is called a carry generate and it produces on carry when both A_i and B_i are one, regardless of the input carry.

P_i is called a carry propagate because it is term associated with the propagation of the carry from C_i to C_{i+1} .

The boolean function for the Carry output of each stage can be given as

$$C_2 = G_1 + P_1 C_1$$

$$\begin{aligned} C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2 (G_1 + P_1 C_1) \\ &= G_2 + P_2 G_1 + P_2 P_1 C_1 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 C_1) \\ &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_1 \end{aligned}$$

From the above Boolean function it can be seen that C_4 does not have to wait for C_3 and C_2 to propagate, in fact C_4 is propagated at the same time as C_2 and C_3 .

Digital Comparators

Here, the Combinational circuit Compares numerical values of two binary numbers and result is presented by the outputs of Combinational Circuits